

基于中间语言与指令模板的联合 DFA 构建及环境验证

姓名：李祥熙 学号：2234412799

2026 年 5 月 8 日

1 实验目的

本实验旨在通过理论与实践结合，掌握基于 σ -DFA 的后端代码生成流程。具体目标包括：

- 理论构建：根据设计的中间语言 (QTAC) 和指令模板 (Q2ARM)，构建 σ -DFA 用于指导代码生成。
- 程序实现：编写程序实现 DFA 的模板匹配逻辑，将中间代码转换为特定架构 (ARM64) 的汇编代码。
- 环境验证：利用类似于阶乘 `fact` 的测试算法生成目标用例的汇编代码，并在真实 ARM 架构系统（鲲鹏环境以及等效 Apple Silicon macOS 环境）中完成汇编、链接并运行，验证编译生成的正确性。

2 实验过程

2.1 实验环境

- 硬件平台：鲲鹏 ARM 服务器 (Linux) / Apple Silicon macOS
- 编程语言：Python 3 (代码生成器实现)、C 语言 (测试驱动)、ARM64 汇编
- 编译工具链：GCC / Clang (用于对象汇编和符号链接)

2.2 实验素材与设定

- 中间代码表示：设计并使用了基于规则的中间代码表示 (QTAC 格式)，实验采用计算阶乘程序的表示。
- 架构指令映射：通过查阅 Q2ARM 的模式规则指导汇编生成。

- **寄存器分配策略**：变量 `n` 使用 `X0`，`a` 使用 `X1`，中间状态量依次映射至 `X2`，`X3...`，从而进行极简的直接分配。

2.3 实现步骤

1. 根据给定的阶乘逻辑编写 QTAC 中间语言，形成 `fact.qtac` 文件。
2. 使用 Python 正则表达式与表驱动模型，设计相应的 DFA 转移结构，编写翻译分析器 `codegen.py` 输出 ARM64 汇编。
3. 针对架构的特殊性（如 macOS ABI 与 Linux ABI 对导出符号的要求不同），进行相关的代码改造调优，同时引出 C 主驱动程序 `main.c`。
4. 分步执行预处理、翻译、编译、符号链接，获得最终产物 `fact_run`。

3 实验内容

3.1 任务一：实现中间代码到目标汇编的代码生成器

根据实验指导的 QTAC 和 Q2ARM 定义，在 Python 实现了翻译的模式匹配逻辑：主要映射逻辑为循环匹配每行指令的规则类型：

- 控制流：识别 `LABEL 1` 输出汇编标签 `1:`，识别 `GOTO` 并输出 `B` 指令；识别带有运算的 `IF...THEN...ELSE` 表达式。
- 二元与单元操作：正则匹配赋值语句 `qd = qs op qt` 并映射到对应的 `ADD`, `SUB`, `MUL`, `SDIV` 或者将数加载到寄存器的 `MOV` 语句。
- 函数返回 `RETURN`：转化为设置返回值与触发调用返回的 `RET` 命令。

在这个过程中增加了一个简单的优化策略来消除冗余转移：当遇到 `IF` 并且 `ELSE 12` 的目的地刚好在代码段的下一行指令时，省略生成 `B 12` 控制指令。

3.2 任务二：系统平台调用约定的适配处理

在将代码生成器生成的汇编应用于实际物理机编译链接时，遇到跨平台的符号调用缺失。具体现象：在 Apple 设备的 macOS 环境调用 GCC（或 Clang 代理层）时，触发链接错误 `_fact` 未定义。解决方案：相比于鲲鹏 Linux 环境生成的 ELF 标准只要导出 `fact`，macOS 下的 Mach-O 二进制 ABI 规定所有的 C 函数导出会含有一个 `'_'` 前缀。因此在汇编导出部分主动同时注册了两者：

$$.global\ fact \quad \wedge \quad .global\ _fact \quad (1)$$

这确保了同一个转换结果在两种平台上都能被标准 C 框架正常使用。

4 实验结果

4.1 功能演示与结果验证

[illegible]

图 1: 代码编译链接步骤与终端执行结果输出

结果分析:

- 通过生成器产生的 ARM64 汇编语法严格正确
- 主程序利用给定的常量 $n=5$ 和初始值 $a=1$ 调用模块，得到结果 120，说明逻辑流在条件判别以及迭代调用时执行精确。
- 返回后无段栈异常被爆裂，表明对栈指针对齐规则无破坏及未发生系统级运行崩溃 (SIGSEGV)。

5 总结

本实验成功完成了基于理论多语言 DFA 思维在后端代码实际实现上的全流程测试：

- 完成并熟悉了从定义虚拟指令代码与实际物理 ISA（指令集架构）之间的表述对应关系。
- 完成了一套实用的 Python 汇编代码生发驱动，正确通过了 `fact`（阶乘）功能原型的所有生成。
- 深入分析并适配了 macOS (Apple ARM64) 和 Linux (鲲鹏 ARM) 的 C 函数链接 ABI 特性（下划线修饰）。

附录：实验源代码

A.1 代码生成器：codegen.py

```
import sys
import re

def main():
    if sys.stdin.isatty():
        print("Please pipe the qtac file into this script. e.g. python
              codegen.py < fact.qtac")
        return
    qtac_code = sys.stdin.read()

    # Preprocess
    qtac_code = qtac_code.replace('\n', ' ')
    instructions = [i.strip() for i in qtac_code.split(';') if i.strip()
                    ()]

    reg_map = {
        'n': 'X0',
        'a': 'X1',
        't1': 'X2',
        't2': 'X3',
        't3': 'X4',
        't4': 'X5'
    }

    def get_op(op):
        if op == '+': return 'ADD'
        if op == '-': return 'SUB'
        if op == '*': return 'MUL'
        if op == '/': return 'SDIV'
        return 'UNKNOWN'

    def get_jrop(op):
        mapping = {
            '<': 'B.LT', '<=': 'B.LE', '>': 'B.GT', '>=': 'B.GE', '==':
            'B.EQ', '!=': 'B.NE'
        }
        return mapping.get(op, 'B.EQ')
```

```

asm = []
asm.append(".global fact")
asm.append(".global _fact")
asm.append("fact:")
asm.append("_fact:")

i = 0
while i < len(instructions):
    inst = instructions[i]

    # LABEL l
    m = re.match(r'^LABEL\s+(\w+)\$', inst)
    if m:
        asm.append(f"{m.group(1)}:")
        i += 1
        continue

    # GOTO l
    m = re.match(r'^GOTO\s+(\w+)\$', inst)
    if m:
        asm.append(f"\tB {m.group(1)}")
        i += 1
        continue

    # RETURN q
    m = re.match(r'^RETURN\s+(\w+)\$', inst)
    if m:
        var = m.group(1)
        reg = reg_map.get(var, var)
        asm.append(f"\tMOV X0, {reg}\n\tRET")
        i += 1
        continue

    # IF qs rop qt THEN l1 ELSE l2
    m = re.match(r'^IF\s+(\w+)\s*([<=>!=]+)\s*(\w+)\s+THEN\s+(\w+)\s'+
        '+ELSE\s+(\w+)\$', inst)
    if m:
        qs, rop, qt, l1, l2 = m.groups()
        rqs = reg_map.get(qs, qs)

```

```

# Check if qt is immediate or register
is_imm = qt.isdigit()
rqt = f"#{qt}" if is_imm else reg_map.get(qt, qt)

# check next instruction to optimize conditional jump
next_inst = instructions[i+1] if i+1 < len(instructions)
else ""
m_next = re.match(r'^LABEL\s+(\w+)\$', next_inst)
if m_next and m_next.group(1) == l2:
    # IF qs < qt THEN l1 ELSE l2; LABEL l2
    asm.append(f"\tCMP {rqs}, {rqt}")
    asm.append(f"\t{get_jrop(rop)} {l1}")
    asm.append(f"{l2}:")
    i += 2
    continue

asm.append(f"\tCMP {rqs}, {rqt}")
asm.append(f"\t{get_jrop(rop)} {l1}")
asm.append(f"\tB {l2}")
i += 1
continue

# qd = qs op qt or qd = qs op k
m = re.match(r'^(\w+)\s*=\s*(\w+)\s*([\+|-|*|/])\s*(\w+)\$',
inst)
if m:
    qd, qs, op, qt = m.groups()
    rqd = reg_map.get(qd, qd)
    rqs = reg_map.get(qs, qs)
    is_imm = qt.isdigit()
    rqt = f"#{qt}" if is_imm else reg_map.get(qt, qt)
    aop = get_op(op)
    asm.append(f"\t{aop} {rqd}, {rqs}, {rqt}")
    i += 1
    continue

# qd = qs (move) or qd = k
m = re.match(r'^(\w+)\s*=\s*(\w+)\$', inst)
if m:
    qd, qs = m.groups()
    rqd = reg_map.get(qd, qd)

```

```

        is_imm = qs.isdigit()
        rqs = f"#{qs}" if is_imm else reg_map.get(qs, qs)
        asm.append(f"\tMOV {rqd}, {rqs}")
        i += 1
        continue

    asm.append(f"\t// UNKNOWN: {inst}")
    i += 1

    for line in asm:
        print(line)

if __name__ == '__main__':
    main()

```

A.2 中间代码验证原件: fact.qtac

```

LABEL 10;
t1=2;
IF n<t1 THEN l1 ELSE 12;
LABEL 11;
RETURN a;
GOTO 13;
LABEL 12;
t2=n-1;
t3=n*a;
n=t2;
a=t3;
GOTO 10;
LABEL 13;

```

A.3 跨平台测试主引导代码: main.c

```

#include <stdio.h>

extern long long fact (long long n, long long a);

int main ()
{
    long long n = 5;

```



```
long long a = 1;
long long result = fact (n, a);
printf ("fact(%lld, %lld) = %lld\n", n, a, result);
return 0;
}
```

A.4 构建与运行命令

5.0.1 报告中展示的自动化脚本 run.sh

[illegible]

```
rm fact_asm.o && rm fact_asm.s && rm fact_run && rm main.o  
echo ""
```

5.0.2 手动运行

1. 编译中间层语言为底层汇编

```
python codegen.py < fact.qtac > fact_asm.s
```

2. 汇编转换并生成对象文件

```
gcc -c fact_asm.s -o fact_asm.o
```

```
gcc -c main.c -o main.o
```

3. 链接并执行验证结果

```
gcc fact_asm.o main.o -o fact_run
```

```
./fact_run
```